AutoML Gap Analysis

What your current product already does, what it does not, and what to build first — checked against the actual code in `AutoAI-main`, not against a feature list.

6 October 2026
26 modules reviewed
Evidence: working tree, not last commit

How to read this. Every rating comes from reading the code in the AutoAI-main working tree on 6 October 2026, including the ~25 new files and 59 modified files that are not yet committed. Where no code could be found for a capability it is marked *Not available* rather than assumed. Section 2 lists all 26 modules with the reasoning behind each rating.

01 Executive summary

READY TO REUSE	HALF BUILT	NOT THERE AT ALL	REUSABLE CODE
3	**12**	**11**	**~60%**
of 26 modules work today	exist but need work	must be built from zero	of the data engine already written

The short version

You are much further along than your own feature list suggests — but in a narrow strip. Your product is **very strong at one thing**: taking a database connection and turning it into a correct, reconciled dataset. That is the hardest part of this whole platform and it is already working.

What is missing is almost everything *around* that strip. There is no sign-in, no users, no audit trail, no business glossary, no monitoring, no catalog. The engine is good; the enterprise shell is not built yet.

Three things that surprised me — in a good way

- **Pipeline saving and versioning is built.** Your earlier plans called this the missing foundation. It now exists: `pipeline_definitions`, `pipeline_versions`, `pipeline_runs` tables, 13 working API endpoints, plus Save / Clone / Restore / Compare / Run-history.
- **Alembic is set up.** `backend/alembic.ini` and `backend/migrations/` exist. Database changes are now manageable.
- **Profiling is genuinely deep.** Nulls, duplicates, outliers, datatype problems and quality scores each have their own backend service *and* their own screen.

Three things on your list that I could not find

- **Documentation generation.** You listed it as a current capability. I searched the whole backend for data-dictionary or documentation code and found none. Marked *Not Available*.
- **Most database connectors.** You listed MySQL, Oracle and Snowflake. The code supports **PostgreSQL and SQL Server only**. Excel and API are not there either.
- **Any login at all.** There are login and register *pages*, but no password handling, no tokens, no user table. They are empty shells.

THE ONE DECISION THAT MATTERS MOST

Build sign-in before anything else. It is not exciting and no customer will praise it, but five of your most valuable future features — audit logs, access control, approvals, ownership, governance — cannot exist without knowing who the user is. Every week you delay it, you add more code that will need rewriting to become user-aware.

Where you stand on your own nine-step vision

Connect data → Ingest data → Store data → Understand data → Improve data → Build trusted dataset → Govern data → Monitor data → Prepare for analytics & AI

Green working now Amber partly working Red not started. Steps 4 and 6 — understanding data and building a trusted dataset — are your moat. Steps 7 and 8 are where enterprise buyers will say no.

02 Module-by-module comparison

01 Data Source Connectivity

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

PostgreSQL and SQL Server, both with read-only enforcement and timeouts. CSV upload works through an upload endpoint.

WHAT YOU CAN REUSE

The whole connection service. It is written on SQLAlchemy, so adding a database is mostly a connection string and a dialect check.

WHAT IS MISSING

MySQL, Oracle, Snowflake, Excel and REST APIs. None of them appear anywhere in the code.

> **Why I rated it this way.** Your list said eight sources. The file that builds connections defines exactly two constants: POSTGRESQL and MSSQL. MySQL is close to free because it uses the same library. Oracle and Snowflake each need their own driver and testing.

02 Data Ingestion Hub

Not available | High effort | Postpone

WHAT EXISTS TODAY

Nothing. Data is read from the source and written straight out as a dataset.

WHAT YOU CAN REUSE

The connection service and the job-status pattern already used by the ETL build.

WHAT IS MISSING

The entire layered idea — landing zone, raw, processed, business. There is no concept of storing data in stages.

> **Why I rated it this way.** This is the biggest single gap and the most expensive. It is not a screen; it is a change to how data is stored. Worth doing eventually because it makes failures recoverable, but it will not win a demo.

03 Data Storage Architecture

Not available | High effort | Postpone

WHAT EXISTS TODAY

Nothing. There is one PostgreSQL database for application data and dataset files on disk.

WHAT YOU CAN REUSE

Nothing meaningful.

WHAT IS MISSING

Lake, warehouse and layered storage, plus the metrics to show what sits where.

> **Why I rated it this way.** This follows directly from the ingestion layers above. Building the screen without the layers would be a picture of something that does not exist.

04 Data Landscape Explorer

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

A database explorer screen, a relationship graph built from declared foreign keys, measured join match rates and a join verifier.

WHAT YOU CAN REUSE

The join graph and verifier are solid and tested. The explorer component works.

WHAT IS MISSING

Business domains as a grouping concept, and relationships that span more than one source system.

> **Why I rated it this way.** Everything works within a single database connection. There is no way to join a SQL Server table to a PostgreSQL table, because the connection service handles one connection at a time.

05 AI Discovery Center

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

Schema reading, metadata extraction, relationship discovery and a spec generator that turns a request into a dataset plan.

WHAT YOU CAN REUSE

All of it. The schema reader and the catalog builder that strips data before sending anything to the AI.

WHAT IS MISSING

Generated business descriptions for tables and columns. The AI reads the schema but does not write back explanations of what things mean.

> **Why I rated it this way.** The discovery half is strong. The describe-it-in-business-language half does not exist, which is also why the Documentation Center cannot be built yet.

06 Data Health Center

Fully available | Low effort | Reuse

WHAT EXISTS TODAY

Column profiling from the live database, plus dedicated analysis for nulls, duplicates, outliers and datatype problems, a quality score card, and a rules engine with six rule types and severities.

WHAT YOU CAN REUSE

Everything. Backend services and six matching React components.

WHAT IS MISSING

Freshness checking — nothing measures how old the data is.

> **Why I rated it this way.** This is the most complete module in the product. It has its own service, its own router, its own rules engine and its own screens. Only freshness is absent, and that needs scheduling to be meaningful.

07 Dataset Recommendation Engine

Not available | Medium effort | Build new

WHAT EXISTS TODAY

The copilot can build a dataset from a written request, which is related but not the same thing.

WHAT YOU CAN REUSE

The spec generator and the join graph — a recommendation is essentially a pre-written spec with a quality estimate.

WHAT IS MISSING

Proactive suggestions. Nothing looks at a schema and says "you could build a Customer 360 from these five tables".

> **Why I rated it this way.** Today the user must know what they want. The gap is the step before that. It is medium effort because the hard part — knowing which joins are valid — is already solved.

08 Visual Dataset Builder

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

A multi-table builder component, join preview, the dataset compiler with correct grain handling, and table preview.

WHAT YOU CAN REUSE

The compiler is the crown jewel. 43 KB that gets joins right.

WHAT IS MISSING

A visual drag-and-drop canvas, filter management and a preview panel that stays visible while you work.

> **Why I rated it this way.** The engine underneath is excellent; the interface is a form rather than a canvas. That is a front-end job, not a rethink.

09 Transformation Studio

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

Two separate systems. A cleaning service with four step types, saved per pipeline version with preview and history. And a transform server with fourteen operations on a different screen.

WHAT YOU CAN REUSE

Both. The fourteen operations are clean, separate functions, easy to move.

WHAT IS MISSING

They are not the same screen. Standardisation, merge columns, split columns and derived columns are not in the saved cleaning pipeline.

> **Why I rated it this way.** This is the clearest case of good work in two places. Merging them is mostly plumbing and gives you a studio with eighteen operations instead of two screens with four and fourteen.

10 SQL Copilot

Partly available | Low effort | Enhance

WHAT EXISTS TODAY

A SQL builder that composes queries from verified join paths only, with validation before anything runs.

WHAT YOU CAN REUSE

All of it, including the safety checks that refuse anything other than a SELECT.

WHAT IS MISSING

Plain-English explanation of the generated query, an execution plan, and a warning about what the query will drop.

> **Why I rated it this way.** Low effort and high perceived value. The information needed for the impact warning — which joins drop rows — is already computed by the compiler.

11 Feature Engineering Center

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

Calculated columns through the ETL agent and the transform screen.

WHAT YOU CAN REUSE

The calculated-column mechanism.

WHAT IS MISSING

Suggestions. Nothing proposes features or estimates how useful one would be.

> **Why I rated it this way.** You can create a feature if you know what you want. Nothing recommends one, and nothing warns when a feature leaks the answer — which is the mistake that quietly ruins models.

12 Validation Center

Partly available | Low effort | Enhance

WHAT EXISTS TODAY

A spec validator, the quality rules engine with pass/fail, and reconciliation as a hard gate.

WHAT YOU CAN REUSE

All three. Together they are a stronger validation story than most tools have.

WHAT IS MISSING

The five named scores presented together — completeness, consistency, freshness, integrity and overall.

> **Why I rated it this way.** The measurements mostly exist; they are scattered. Pulling them into one screen is a few days and makes the strength visible.

13 Explainability Center

Partly available | Low effort | Enhance

WHAT EXISTS TODAY

An explainability card component, and an agent_decisions table storing the agent name, decision type, decision text and confidence score.

WHAT YOU CAN REUSE

The table is exactly the right shape. It is already being written to.

WHAT IS MISSING

A full screen that answers why a table, join, transformation or row removal was chosen.

> **Why I rated it this way.** This is the cheapest high-value item in the whole analysis. The data is being recorded already — it just is not being shown.

14 Documentation Center

Not available | Medium effort | Build new

WHAT EXISTS TODAY

Nothing found.

WHAT YOU CAN REUSE

The profiler output and the compiler plan contain almost everything a data dictionary needs.

WHAT IS MISSING

The data dictionary, dataset documentation, business glossary and column descriptions.

> **Why I rated it this way.** You listed this as a current capability, so I searched specifically for it across the whole backend and found no matching code. Marking it available would have been wrong.

15 Data Lineage

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

A lineage module and a lineage graph component, returned with every build.

WHAT YOU CAN REUSE

Both. The lineage is generated from the compiled plan, so it is accurate rather than guessed.

WHAT IS MISSING

Anything beyond one build. There is no raw or clean layer to trace through, and no link onward to reports or models.

> **Why I rated it this way.** Current lineage answers "which tables made this dataset". The vision asks for source through to model. The second half needs the storage layers and a consumer registry, neither of which exists.

16 Pipeline Builder

Fully available | Medium effort | Reuse

WHAT EXISTS TODAY

Three database tables, thirteen API endpoints and three React components. Save, list, update, delete, clone, version history, compare versions, restore, cleaning steps per version, run history and run.

WHAT YOU CAN REUSE

All of it. This was the missing foundation in earlier plans and it is now built.

WHAT IS MISSING

A dedicated page in the app. The components exist but are not reachable from the navigation.

> **Why I rated it this way.** Marked fully available because the backend genuinely is. The front end needs a route and a page, which is small work on top of real foundations.

17 Monitoring & Observability

Not available | High effort | Build new

WHAT EXISTS TODAY

Nothing. No scheduler, no metrics store, no alerting. A search for freshness, alert and drift found only unrelated comments.

WHAT YOU CAN REUSE

The quality rules engine could become the checking half once something runs it on a schedule.

WHAT IS MISSING

Scheduling, metrics history, alert rules, alert delivery, freshness and schema-change detection.

> **Why I rated it this way.** Three missing pieces, not one. This is why it is high effort despite sounding simple.

18 Root Cause Analysis

Not available | High effort | Postpone

WHAT EXISTS TODAY

Nothing.

WHAT YOU CAN REUSE

Nothing yet.

WHAT IS MISSING

The whole module.

> **Why I rated it this way.** Cannot be built before monitoring, because there is no event history to analyse. Building it earlier would produce a screen with nothing to show.

19 Data Reconciliation

Fully available | Low effort | Reuse

WHAT EXISTS TODAY

A reconciler module that compares source and target totals with tolerance, raises a specific error on failure, and a report component in the UI.

WHAT YOU CAN REUSE

All of it. This is your strongest differentiator and it is finished.

WHAT IS MISSING

Row-level detail of which specific records are missing or mismatched.

> **Why I rated it this way.** It works and it blocks a bad build from publishing. Most competing tools report a variance; yours refuses to release. That is a better story — tell it louder.

20 Impact Analysis

Not available | High effort | Postpone

WHAT EXISTS TODAY

Nothing.

WHAT YOU CAN REUSE

The lineage module is the starting point.

WHAT IS MISSING

A dependency graph covering datasets, pipelines, reports and downstream systems.

> **Why I rated it this way.** Impact analysis is lineage read backwards. It needs lineage to be complete first, and it needs a register of consumers that does not exist.

21 Agent Control Center

Partly available | Medium effort | Enhance

WHAT EXISTS TODAY

One large ETL agent with eleven tools, an agent_runs table, and an agent_decisions table with confidence scores.

WHAT YOU CAN REUSE

The decision and run tables are the right foundation. The eleven tools map cleanly onto separate agents.

WHAT IS MISSING

Agents as separate named things, and any screen showing their status.

> **Why I rated it this way.** Today it is one agent that does everything. The vision shows six to eight specialists. This is mostly reorganisation plus a screen, not new intelligence.

22 Dataset Marketplace

Not available | Medium effort | Build new

WHAT EXISTS TODAY

A saved-recipes feature that stores up to ten entries in the browser only. It disappears if the user clears their browser.

WHAT YOU CAN REUSE

The pipeline storage tables are most of a marketplace already — they hold definitions, versions and ownership.

WHAT IS MISSING

Search, sharing, cloning between users, certification status and usage counts.

> **Why I rated it this way.** Closer than it looks. Saved pipelines plus a shared flag and a certified flag is roughly the whole feature. Low value until a customer has many datasets, though.

23 Business Knowledge Layer

Not available | Medium effort | Build new

WHAT EXISTS TODAY

Nothing. A search for glossary and business-definition code found no matches.

WHAT YOU CAN REUSE

The catalog builder that assembles what the AI sees — definitions would be added there.

WHAT IS MISSING

The definitions table, versioning, ownership, and the step that feeds definitions to the AI before the schema.

> **Why I rated it this way.** This is the feature that makes your AI sound like it knows the customer business rather than just their column names. For a bank, "active customer" has one specific meaning, and today your product cannot know it.

24 Governance & Compliance

Not available | High effort | Build new

WHAT EXISTS TODAY

Nothing. No audit table, no roles, no policy code. There is no sign-in either — the login and register pages have no backend behind them.

WHAT YOU CAN REUSE

Nothing.

WHAT IS MISSING

Authentication, users, roles, permissions, audit logging, policies, sensitive-data detection and compliance reporting.

> **Why I rated it this way.** Blocked at the root. Without knowing who the user is, an audit log cannot record who did something. Start with sign-in; the rest becomes possible afterwards.

25 Data Contracts

Not available | Medium effort | Build new

WHAT EXISTS TODAY

Nothing for source schemas. There is a contract validator in the training code, but it checks generated Python scripts, not data schemas — different thing entirely.

WHAT YOU CAN REUSE

The schema reader already captures everything a contract needs to record.

WHAT IS MISSING

Storing an expected schema, comparing it on each load, and reporting added, removed, renamed or retyped columns with their downstream impact.

> **Why I rated it this way.** Medium effort and high protective value. The silent version of this problem — a column declared one type and delivered another — is exactly the class of bug that cost you time in this codebase already.

26 Operations History

Partly available | Low effort | Enhance

WHAT EXISTS TODAY

Three sources of truth already recording history: agent_decisions with confidence scores, pipeline_runs with status and timing, and cleaning rules stamped with who created them.

WHAT YOU CAN REUSE

All three tables. The data is being written today.

WHAT IS MISSING

One unified timeline, and before/after snapshots of what each operation changed.

> **Why I rated it this way.** A quick win. Most of the work is a query across three tables and a screen. Before/after snapshots need a little more, since current operations do not save a sample of the prior state.

03 What you can reuse right now

These are finished pieces. Do not rewrite them — wrap them in new screens.

COMPONENT	WHERE IT LIVES	WHY IT IS WORTH KEEPING
Dataset compiler	`etl_copilot/dataset_compiler.py`	43 KB of logic that gets joins right. It handles fan-out — the bug where joining a loan to its 39 repayments makes the loan amount look 39× bigger. Most teams never solve this properly.
Reconciliation gate	`etl_copilot/reconciler.py`	Compares source totals to output totals and refuses to release the dataset if they disagree. This is the single most sellable feature you have.
Join verifier	`etl_copilot/join_verifier.py`	Measures how well two tables actually join on real data instead of trusting the column names.
Source profiler	`services/data_profiler.py`	Column-level profiling straight from the database. Feeds the health screens and the quality rules.
Quality rules engine	`services/quality_rules.py`	Six rule types with pass/fail and severity. Already stores rules per pipeline version.
Pipeline storage & versioning	`routers/pipelines.py`	Save, list, update, delete, clone, version history, compare two versions, restore, run history, run. Thirteen endpoints, all working.
ETL agent with its privacy wall	`services/etl_agent.py`	111 KB, 11 tools, and a tested guarantee that no customer row ever reaches the AI. That test suite is a sales asset.
14 transform operations	`services/transform_mcp_server.py`	Fill nulls, drop duplicates, convert types, encode, normalise and more. Already written as clean, separate functions.
The whole React component set	`src/components/etl-copilot/`	25 components including profiling, cleaning, quality, lineage and version history. Your UI library is further along than your backend in places.

04 Half-built — exists, needs work

MODULE	WHAT IS THERE	WHAT IS MISSING	EFFORT
Data Source Connectivity	PostgreSQL and SQL Server, both with read-only enforcement and timeouts. CSV upload works through an upload endpoint.	MySQL, Oracle, Snowflake, Excel and REST APIs. None of them appear anywhere in the code.	Medium
Data Landscape Explorer	A database explorer screen, a relationship graph built from declared foreign keys, measured join match rates and a join verifier.	Business domains as a grouping concept, and relationships that span more than one source system.	Medium
AI Discovery Center	Schema reading, metadata extraction, relationship discovery and a spec generator that turns a request into a dataset plan.	Generated business descriptions for tables and columns. The AI reads the schema but does not write back explanations of what things mean.	Medium
Visual Dataset Builder	A multi-table builder component, join preview, the dataset compiler with correct grain handling, and table preview.	A visual drag-and-drop canvas, filter management and a preview panel that stays visible while you work.	Medium
Transformation Studio	Two separate systems. A cleaning service with four step types, saved per pipeline version with preview and history. And a transform server with fourteen operations on a different screen.	They are not the same screen. Standardisation, merge columns, split columns and derived columns are not in the saved cleaning pipeline.	Medium
SQL Copilot	A SQL builder that composes queries from verified join paths only, with validation before anything runs.	Plain-English explanation of the generated query, an execution plan, and a warning about what the query will drop.	Low
Feature Engineering Center	Calculated columns through the ETL agent and the transform screen.	Suggestions. Nothing proposes features or estimates how useful one would be.	Medium
Validation Center	A spec validator, the quality rules engine with pass/fail, and reconciliation as a hard gate.	The five named scores presented together — completeness, consistency, freshness, integrity and overall.	Low
Explainability Center	An explainability card component, and an agent_decisions table storing the agent name, decision type, decision text and confidence score.	A full screen that answers why a table, join, transformation or row removal was chosen.	Low
Data Lineage	A lineage module and a lineage graph component, returned with every build.	Anything beyond one build. There is no raw or clean layer to trace through, and no link onward to reports or models.	Medium
Agent Control Center	One large ETL agent with eleven tools, an agent_runs table, and an agent_decisions table with confidence scores.	Agents as separate named things, and any screen showing their status.	Medium

MODULE	WHAT IS THERE	WHAT IS MISSING	EFFORT
Operations History	Three sources of truth already recording history: agent_decisions with confidence scores, pipeline_runs with status and timing, and cleaning rules stamped with who created them.	One unified timeline, and before/after snapshots of what each operation changed.	Low

05 Not there at all — build from zero

MODULE	EVIDENCE I LOOKED FOR	EFFORT	ADVICE
Data Ingestion Hub	Nothing. Data is read from the source and written straight out as a dataset.	High	Postpone
Data Storage Architecture	Nothing. There is one PostgreSQL database for application data and dataset files on disk.	High	Postpone
Dataset Recommendation Engine	The copilot can build a dataset from a written request, which is related but not the same thing.	Medium	Build new
Documentation Center	Nothing found.	Medium	Build new
Monitoring & Observability	Nothing. No scheduler, no metrics store, no alerting. A search for freshness, alert and drift found only unrelated comments.	High	Build new
Root Cause Analysis	Nothing.	High	Postpone
Impact Analysis	Nothing.	High	Postpone
Dataset Marketplace	A saved-recipes feature that stores up to ten entries in the browser only. It disappears if the user clears their browser.	Medium	Build new
Business Knowledge Layer	Nothing. A search for glossary and business-definition code found no matches.	Medium	Build new
Governance & Compliance	Nothing. No audit table, no roles, no policy code. There is no sign-in either — the login and register pages have no backend behind them.	High	Build new
Data Contracts	Nothing for source schemas. There is a contract validator in the training code, but it checks generated Python scripts, not data schemas — different thing entirely.	Medium	Build new

06–08 Sorted by how hard each one is

Quick wins 1–2 sprints

Mostly surfacing data you already collect.

- **Operations history screen.** The `agent_decisions` and `pipeline_runs` tables already hold this. It needs a screen, not a backend.
- **Explainability screen.** `agent_decisions` stores decision text and a confidence score per agent. Show it.
- **Five validation scores.** Completeness, consistency, freshness, integrity and overall — all computable from the profiler output you already have.
- **SQL explanation and impact warning.** You generate the SQL; add a plain-English breakdown and a warning when an inner join will drop rows.
- **MySQL connector.** Same SQLAlchemy pattern as PostgreSQL. Roughly a day.
- **Excel upload.** You already accept CSV. Pandas reads Excel with one line.

Medium 3–6 sprints

- **Sign-in, users and roles.** Standard work, but it touches every endpoint.
- **Documentation Center.** Generate the data dictionary from the profiler and the compiler plan.
- **Business Knowledge Layer.** A table of definitions, versioned and owned, that the AI reads before the schema.
- **Data Contracts.** Store the expected schema; compare on every load; flag added, removed, renamed and retyped columns.
- **Dataset Marketplace.** Your saved pipelines are 80% of this already — it mainly needs sharing, cloning and a certified flag.
- **Transformation Studio consolidation.** You have 4 cleaning steps in one place and 14 transforms in another. Merge them.
- **Agent Control Center.** Split the one big agent into named agents and show their status.
- **Full lineage.** Extend from "inside one build" to source → raw → clean → dataset → report → model.

Large 6+ sprints

- **Ingestion Hub with landing / raw / processed layers.** This is not a screen — it is a new way of storing data. Today you read from the source and write a dataset; there is no middle.
- **Storage architecture.** Follows the ingestion layers. Cannot be faked.
- **Monitoring & observability.** Needs a scheduler, a metrics store and an alert router. You have none of the three.
- **Root cause analysis.** Only useful once monitoring exists and has history to analyse.
- **Impact analysis.** Needs a complete dependency graph across datasets, reports and models.
- **Governance & compliance.** Audit logs, access control, sensitive-data detection, policies. Blocked until sign-in exists.

09 What the MVP should be

THE PRINCIPLE

Do not try to become Databricks. Become **the tool that produces a dataset you can trust, and proves it**. That is a complete, sellable product on its own, and it is the part you are already closest to finishing.

Everything in the MVP list below either already half-exists or makes your existing strength visible. Everything postponed is infrastructure that will not win a demo.

In the MVP

1. **Sign-in, users, roles**
   Not optional. Everything else depends on it.
2. **Connectors: PostgreSQL, SQL Server, MySQL, CSV, Excel**
   Five covers most mid-market buyers. Oracle and Snowflake can wait.
3. **Data Health Center**
   Already built. Just polish.
4. **Landscape Explorer + Dataset Builder**
   Built. Add filters and a persistent preview panel.
5. **Transformation Studio, merged**
   One screen, all 14 operations, before/after on each.
6. **Validation + Reconciliation**
   Your differentiator. Make it loud.
7. **Explainability + Operations History**
   Cheap to build, directly answers "why should I trust this?"
8. **Documentation Center**
   Generated, so it cannot go stale.
9. **Pipeline save, version, run history**
   Backend done. Needs its own page.

Postponed

- **Ingestion Hub & storage layers**
  Big architectural change. No buyer asks for it by name; they ask for the outcomes it enables.
- **Monitoring, alerts, root cause**
  Needs a scheduler and history. Comes after customers are running daily pipelines.
- **Impact analysis**
  Needs the full dependency graph, which needs lineage finished first.
- **Full governance suite**
  Audit logs in Phase 2; policies and sensitive-data detection in Phase 3.
- **Marketplace & catalog**
  Only valuable once there are many datasets to browse. Your first customer will have three.
- **Model training**
  You already said it is not the focus. Keep it that way — it is a different product.

10 Roadmap

Phase 1 — Make what you have sellable

~8–10 weeks | Highest value per week

Nothing here is new invention. It is finishing, connecting and showing work that mostly exists. At the end you have a product you can demo end to end and charge for.

BUILD

- Sign-in, users, roles — first, before anything else
- Pipeline management page on the existing 13 endpoints
- Merged Transformation Studio with before/after
- Documentation Center generated from the profiler
- Explainability and Operations History screens
- Five validation scores
- MySQL and Excel connectors

YOU WILL BE ABLE TO SAY

"Connect your database. We profile it, find the relationships, propose the cleaning, build the dataset, prove the totals match, explain every decision and document it — and you can save, version and re-run the whole thing."

That sentence sells. None of it requires Phase 2 or 3.

Phase 2 — Make it trustworthy for a bank

~10–12 weeks | Unlocks regulated buyers

This phase is what moves you from "useful tool" to "tool a compliance team will approve". Your BFSI and pharma targets will ask for these in the first meeting.

- **Business Knowledge Layer** — the company's own definitions, versioned and owned, read by the AI before the schema. This is the feature that makes your AI sound like it understands *their* business.
- **Audit logs** — who changed what, when and why. Now possible because sign-in exists.
- **Data Contracts** — catch a column being added, removed, renamed or retyped before it breaks a dataset.
- **Agent Control Center** — split the one agent into named agents with status and confidence. Mostly presentation of work already done.
- **Full lineage** — source through to report and model, with clickable steps.
- **Dataset Marketplace** — sharing, cloning and certification on top of saved pipelines.

Phase 3 — Become a platform ~12–16 weeks Only after real usage

Do not start this until customers are running pipelines daily. Every item here is only valuable once there is real activity to observe, and each one is expensive to build against guesses.

- **Ingestion Hub and storage layers** — landing zone, raw, processed, business. The architectural change that makes everything else recoverable and auditable.
- **Monitoring and alerting** — scheduler, metrics store, alert routing, freshness and drift checks.
- **Root cause analysis** — becomes possible once monitoring has history.
- **Impact analysis** — needs the dependency graph that full lineage produces.
- **Governance suite** — policies, access control, sensitive-data detection, compliance reporting.
- **Remaining connectors** — Oracle, Snowflake, REST APIs, SAP, Salesforce. Add them when a customer asks, not before.

ONE HONEST WARNING

Your working tree has **59 modified files and about 25 new files that are not committed**, on top of a commit from 5 October. The pipeline tables, Alembic setup, profiling services and cleaning screens all live only in that uncommitted work.

That is a lot of valuable work sitting on one machine with no backup. Commit it before you plan anything else.

Assessments are based on reading the code in `AutoAI-main` on 6 October 2026, including uncommitted changes. Where I could not find evidence of a feature, it is marked *Not Available* rather than assumed. Effort estimates assume a two-person team and are for planning conversations, not commitments.